Lehrerexemplar · Boss-Kampf 0 · Welt 0 · DS 7

Erwartungshorizont: Die Bibliothek der Akademie

Erwartungshorizont / Bewertungsbogen für _____ Kurs: _____

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten

Erwartete Prüfungsleistungen	I	II	III	BE
a) Übersetzen von Signaturen; Parameter und Argumente; Typprüfung Kernpunkte: public static double durchschnitt(int[] ausleihen); public static String[] neueAuflage(String[] titel, int[] ausleihen, int grenze); markiere(titel: Reihung von Zeichenkette, index: Ganzzahl, zeichen: Zeichen): nichts; Parameter ausleihen, start, ende (Variablen im Methodenkopf) – Argumente werte, 2, 6 (Werte beim Aufruf, die den Parametern der Reihe nach zugewiesen werden); Kommazahl passt ohne Cast nicht in eine Ganzzahl-Variable (Verlust der Nachkommastellen), eine Ganzzahl wird dagegen automatisch in eine Kommazahl umgewandelt.	2	2		
b) Dokumentation des Aufrufs mit Hilfsmethode; Analyse; Laufzeitfehler Kernpunkte: Fünf Einträge (i = 2 bis 6): sterne(7) → s = 2, Rückgabe 2, summe 2; sterne(25) → s = 6, Rückgabe 5, summe 7; sterne(3) → 1, 1, 8; sterne(0) → s nicht belegt, Rückgabe 0, summe 8; sterne(18) → 4, 4, 12; Rückgabewert 12. sterne liefert 5 genau für n ≥ 20; bei sterneSumme(ausleihen, 5, 8) Laufzeitfehler beim Zugriff auf ausleihen[8].	1	3		
c) Wertübergabe und Referenz Kernpunkte: Ausgaben 11 4 · 12 · 0; anzahl ist eine Kopie von n (Wertübergabe), anzahl = 0 wirkt nicht auf n; ausleihen erhält eine Kopie der Referenz und verweist auf dieselbe Reihung wie a, die Änderung von ausleihen[2] wirkt nach außen (7 + 4 = 11); jahresabschluss lenkt nur die lokale Referenz auf eine neue Reihung um – a bleibt unverändert; b = a kopiert die Referenz, b[0] = 0 ändert auch a[0]; jahresabschluss erfüllt die Anforderung nicht, Korrektur durch Überschreiben jedes Elements.	1	1	1	
d) Entwicklung eines Struktogramms Kernpunkte: Kopf mit Signatur; Leer-Abfrage mit „gib −1 zurück“ vor dem ersten Zugriff; maxIndex ← 0; Zählschleife von 1 bis Länge von ausleihen − 1; Vergleich ausleihen[i] > ausleihen[maxIndex] mit echtem > (kleinster Index bei Gleichstand); „gib maxIndex zurück“ nach der Schleife; Notation nach EH §5.	1	2	1	
e) Implementierung der Operation neueAuflage Kernpunkte: Methodenkopf mit Rückgabetyp String[]; Länge vorab bestimmen (anzahlMindestens oder eigene Zählschleife); neue Reihung genau dieser Länge erzeugen; Durchlauf über alle Indizes 0 bis titel.length − 1 mit Vergleich ≥ grenze; eigener Schreibindex k für erg; Rückgabe der Reihung (bei keinem Treffer Länge 0, nicht null); titel und ausleihen werden nur gelesen.	1	3	1	
f) Fehleranalyse und Korrektur der Operation durchschnitt Kernpunkte: Fehler 1: fehlender Rückgabepfad bei leerer Reihung (Compilerfehler „missing return statement“); Fehler 2: Schleife beginnt bei i = 1, ausleihen[0] = 12 fehlt in der Summe; Fehler 3: summe / ausleihen.length ist ganzzahlige Division, die Nachkommastellen gehen verloren; mit Abbildung 1: korrekt 74 / 8 = 9,25, mit beiden Fehlern 62 / 8 → 7 → 7.0; Korrektur mit return 0.0, Start bei 0 und Division als Kommazahl.	1	2	1	
Summe	7	13	4	**/ 24**

AFB-Verteilung: I 7 BE (29 %) · II 13 BE (54 %) · III 4 BE (17 %) – Abitur-Richtwert 30 / 53 / 17 %. Zeitbudget: 24 BE · 2,5 min = 60 min.

Die gewählten Lösungsansätze und -wege müssen nicht mit den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (EH 2025) entsprechend zu bewerten.

Auswertung und Rückmeldung (fließt nicht in die Note ein)

- Auswertung durch die Lehrkraft oder als Selbst-/Peer-Korrektur nach den Kernpunkten dieses Erwartungshorizonts. Für den Papier-Boss werden im Lab keine Punkte vergeben.
- Papier-Compiler zu e): Handschrift unverändert abgetippt, alle 8 Tests grün beim ersten Versuch → die Handschrift „compiliert“; sonst Fehler ins Fehlerlogbuch.
- Besonders würdigen: In e) und f) sind alle Leer- und Index-Abfragen und alle Rückgabepfade korrekt (e: leere Reihung/kein Treffer liefert Länge 0, Schreibindex k; f: return 0.0 für die leere Reihung, Start bei 0, Kommazahl-Division).
- Orientierung (informativ, Abi-Skala): ab 95 % 15 P · 85 % 13 P · 75 % 11 P · 65 % 9 P · 55 % 7 P · 45 % 5 P · 40 % 4 P · 27 % 2 P. Für 24 BE: 23 BE ≈ 15 P, 18 BE ≈ 11 P, 10 BE ≈ 4 P.
- **Zusätzlich im Lab:** Online-Boss „Aufnahmeprüfung: Die Bibliothek der Akademie“ (25 min, automatisch ausgewertet, bester Versuch zählt, besiegt ab 50 %). Ergebnisse: Lehrkraft-Bereich → Kurs → Auswertung (Boss-Ergebnisse).

Musterlösungen, Kernpunkte und typische Fehler

a) Übersetzen von Signaturen; Parameter und Argumente; Typprüfung 4 BE (I 2 · II 2 · III 0)

Musterlösung

(1) `public static double durchschnitt(int[] ausleihen)` · `public static String[] neueAuflage(String[] titel, int[] ausleihen, int grenze)`

(2) markiere(titel: Reihung von Zeichenkette, index: Ganzzahl, zeichen: Zeichen): nichts

(3) Parameter sind die im Methodenkopf deklarierten Variablen ausleihen, start und ende. Argumente sind die beim Aufruf übergebenen Werte werte, 2 und 6. Beim Aufruf erhält der Parameter ausleihen eine Kopie der Referenz in werte (beide verweisen auf dieselbe Reihung), start den Wert 2 und ende den Wert 6. Zuordnung über die Reihenfolge, nicht über den Namen.

(4) durchschnitt liefert eine Kommazahl. Die Zuweisung an eine Ganzzahl-Variable könnte die Nachkommastellen verlieren (z. B. 9,25 → 9) und wird deshalb ohne ausdrücklichen Cast abgelehnt. sterneSumme liefert eine Ganzzahl; sie lässt sich ohne Verlust als Kommazahl darstellen und wird automatisch umgewandelt (s = 10.0). Geprüft mit javac: „possible lossy conversion from double to int“.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenköpfe von durchschnitt und neueAuflage	I	1	☐
Signatur von markiere (Zeichen, nichts)	I	1	☐
Parameter und Argumente benannt und unterschieden	II	1	☐
Begründung über Rückgabetyp und Typumwandlung	II	1	☐

Alternativen / Teilpunkte: Ohne static ist ein Methodenkopf gleichwertig, sofern Rückgabetyp, Name und Parameterliste stimmen; Parameternamen dürfen abweichen, Reihenfolge und Typen nicht. „Formale/aktuelle Parameter“ statt Parameter/Argument ist gleichwertig. Für (4) reicht der Hinweis „double passt nicht ohne Verlust in int, int passt in double“; ein Cast als Lösungsvorschlag ist nicht gefordert.

Typische Fehler

- Rückgabetyp int bzw. String statt int[] bzw. String[] („Reihung von …“ übersehen).
- „nichts“ als Typname in Java statt void bzw. void in der Signatur.
- Parameter und Argumente vertauscht oder nur „die Werte in der Klammer“ genannt.
- „sterneSumme liefert eine Ganzzahl, also wird auch die Zuweisung an double abgelehnt“.

b) Dokumentation des Aufrufs mit Hilfsmethode; Analyse; Laufzeitfehler 4 BE (I 1 · II 3 · III 0)

Musterlösung

Tracetabelle (geprüft durch Ausführen einer instrumentierten Java-Fassung von Abbildung 3):

i	Aufruf von sterne	s (in sterne)	Rückgabe von sterne	summe
vor der Schleife	–	–	–	0
2	sterne(7)	2	2	2
3	sterne(25)	6	5	7
4	sterne(3)	1	1	8
5	sterne(0)	–	0	8
6	sterne(18)	4	4	12

Rückgabewert von sterneSumme(ausleihen, 2, 6): **12**.

sterne liefert 5 **genau für n ≥ 20**: Für n > 0 ist s = n / 5 + 1 (Division ohne Rest). s ≥ 5 gilt genau dann, wenn n / 5 ≥ 4, also n ≥ 20. Für n = 20 bis 24 ist s = 5 und wird mit „gib s zurück“ geliefert; ab n = 25 ist s > 5 und es wird auf 5 begrenzt. Für n ≤ 19 ist s ≤ 4 (z. B. sterne(19) = 4).

sterneSumme(ausleihen, 5, 8): Die Zählschleife läuft einschließlich ende. Für i = 5, 6, 7 wird summe auf 0, 4 und 6 erhöht. Bei i = 8 wird auf ausleihen[8] zugegriffen; die Reihung hat aber nur die Indizes 0 bis 7. Es kommt zu einem Laufzeitfehler (in Java ArrayIndexOutOfBoundsException), die Operation liefert keinen Wert.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Einträge für i = 2 und i = 3 (inkl. Begrenzung: s = 6, Rückgabe 5)	I	1	☐
Einträge i = 4 bis 6 (s bei n = 0 nicht belegt) und Rückgabewert 12	II	1	☐
sterne liefert 5 genau für n ≥ 20, begründet mit der Division ohne Rest	II	1	☐
Laufzeitfehler bei i = 8 (Index außerhalb 0 bis 7) erläutert	II	1	☐

Alternativen / Teilpunkte: Folgefehler werden nicht erneut abgezogen. Eine Begründung über Beispiele (19 → 4, 20 → 5, 24 → 5, 25 → 6 → 5) ist gleichwertig, wenn klar wird, dass s mit n nicht kleiner wird. Der Name der Ausnahme ist nicht gefordert.

Typische Fehler

- n / 5 als Kommazahl gerechnet (sterne(7) = 2,4).
- Begrenzung übersehen: sterne(25) liefert 6.
- Für sterne(0) wird s = 1 eingetragen, obwohl die Operation bereits mit „gib 0 zurück“ endet.
- „bis ende“ als „bis ende − 1“ gelesen (nur vier Einträge, Rückgabe 8).
- „Bei i = 8 wird 0 addiert“ statt Laufzeitfehler.

c) Wertübergabe und Referenz 3 BE (I 1 · II 1 · III 1)

Musterlösung

Ausgaben (geprüft durch Ausführen): **11 4 · 12 · 0**.

1. Ausgabe: Der Parameter ausleihen erhält eine Kopie der Referenz in a – beide verweisen auf dieselbe Reihung. ausleihen[2] = 7 + 4 = 11 verändert also die Reihung von a. anzahl ist dagegen eine Kopie des Wertes von n (Wertübergabe); anzahl = 0 ändert nur diese Kopie, n bleibt 4.

2. Ausgabe: In jahresabschluss wird nur die lokale Variable ausleihen auf eine neue Reihung umgelenkt. a verweist weiterhin auf die alte Reihung, a[0] ist 12.

3. Ausgabe: int[] b = a kopiert die Referenz, nicht die Reihung. b[0] = 0 ändert die Reihung, auf die auch a verweist – a[0] ist 0.

Beurteilung: jahresabschluss setzt die Ausleihzahlen des Aufrufers nicht auf 0 – die neue Reihung ist nach dem Ende der Operation nicht mehr erreichbar. Die Operation ist wirkungslos. Korrekt ist, jedes Element der übergebenen Reihung zu überschreiben:

```
public static void jahresabschluss(int[] ausleihen) {
    for (int i = 0; i < ausleihen.length; i++) {
        ausleihen[i] = 0;
    }
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Drei Ausgaben 11 4 · 12 · 0	I	1	☐
Begründung mit Wertübergabe (anzahl) und Referenz (ausleihen, b)	II	1	☐
Beurteilung von jahresabschluss und korrigierte Implementierung	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: Rückgabe einer neuen Reihung (public static int[] jahresabschluss(int[] ausleihen) { return new int[ausleihen.length]; }), dann muss der geänderte Aufruf a = jahresabschluss(a); angegeben werden. Eine for-each-Schleife (for (int x : ausleihen) x = 0;) ist keine Korrektur.

Typische Fehler

- „11 0“: Annahme, anzahl = 0 verändere n.
- „0“ als zweite Ausgabe: jahresabschluss wirke wie gewünscht.
- „12“ als dritte Ausgabe: b als Kopie der Reihung statt der Referenz verstanden.
- Korrektur mit for-each-Schleife, die nur die Kopie x verändert.

d) Entwicklung eines Struktogramms 4 BE (I 1 · II 2 · III 1)

Musterlösung

Musterlösung (in Java übersetzt und geprüft: Abbildung 1 → 3, leere Reihung → −1, [4, 9, 9, 2] → 1):

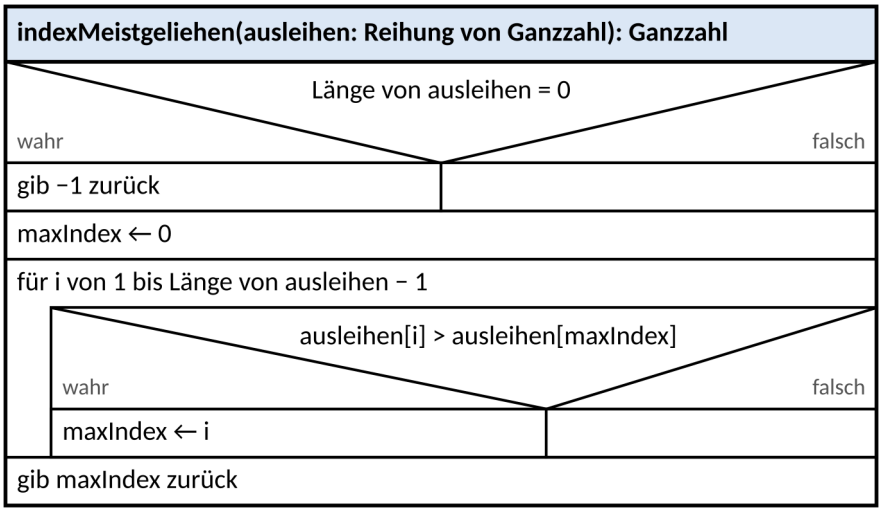


Abbildung EH.1: Struktogramm der Operation indexMeistgeliehen

Zum Vergleich in Java:

```
public static int indexMeistgeliehen(int[] ausleihen) {
    if (ausleihen.length == 0) {
        return -1;
    }
    int maxIndex = 0;
    for (int i = 1; i < ausleihen.length; i++) {
        if (ausleihen[i] > ausleihen[maxIndex]) {
            maxIndex = i;
        }
    }
    return maxIndex;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Kopf mit Signatur, Startwert maxIndex ← 0 und Rückgabe nach der Schleife	I	1	☐
Schleife über alle weiteren Indizes mit Vergleich gegen das bisherige Maximum	II	1	☐
Echter Vergleich > (kleinster Index bei Gleichstand); Notation nach EH §5	II	1	☐
Leer-Abfrage vor dem ersten Zugriff mit Rückgabe −1	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: zusätzliche Variable max für den bisher größten Wert; Schleife ab 0; Leer-Abfrage als Verzweigung, deren falsch-Zweig den restlichen Algorithmus enthält; Startwert max ← 0 (zulässig, da Ausleihzahlen nicht negativ sind). Java statt Struktogramm: höchstens 3 BE.

Typische Fehler

- ≥ statt > (liefert bei Gleichstand den letzten Index).
- Leer-Abfrage fehlt oder steht erst nach dem Zugriff auf ausleihen[0].
- Rückgabe des Wertes (25) statt des Index (3).
- „gib maxIndex zurück“ innerhalb der Schleife.
- Java-Syntax im Struktogramm (=, ++, length) statt EH-Notation; leerer falsch-Zweig fehlt.

e) Implementierung der Operation neueAuflage 5 BE (I 1 · II 3 · III 1)

Musterlösung

Musterlösung (Java, mit javac und den Papier-Compiler-Tests geprüft: 8/8 grün):

```
public static String[] neueAuflage(String[] titel, int[] ausleihen, int grenze) {
    int anzahl = anzahlMindestens(ausleihen, grenze);
    String[] erg = new String[anzahl];
    int k = 0;
    for (int i = 0; i < titel.length; i++) {
        if (ausleihen[i] >= grenze) {
            erg[k] = titel[i];
            k++;
        }
    }
    return erg;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf; Rückgabe der neuen Reihung in jedem Pfad	I	1	☐
Länge bestimmen (Hilfsmethode oder Zählschleife) und neue Reihung passender Länge erzeugen	II	1	☐
Durchlauf über alle Indizes mit Vergleich ≥ grenze	II	1	☐
Eigener Schreibindex k für erg (nicht i)	II	1	☐
Randfälle: leere Reihungen und kein Treffer liefern Länge 0 ohne Laufzeitfehler; Originale unverändert	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (beide ebenfalls 8/8 Tests grün): (1) eigene Zählschleife statt anzahlMindestens; (2) Treffer zunächst in eine Zwischenreihung der Länge titel.length schreiben und danach in eine Reihung der Länge k kopieren. Struktogramm statt Java: höchstens 3 BE (vgl. Abbildung EH.2). Kleinere Syntaxfehler führen nicht zu Abzug, wenn die Logik eindeutig ist – beim Papier-Compiler zählen sie aber!

Alternative (1): eigene Zählschleife

```
public static String[] neueAuflage(String[] titel, int[] ausleihen, int grenze) {
    int anzahl = 0;
    for (int x : ausleihen) {
        if (x >= grenze) {
            anzahl++;
        }
    }
    String[] erg = new String[anzahl];
    int k = 0;
    for (int i = 0; i < titel.length; i++) {
        if (ausleihen[i] >= grenze) {
            erg[k] = titel[i];
            k++;
        }
    }
    return erg;
}
```

Alternative (2): Zwischenreihung, danach kopieren

```
public static String[] neueAuflage(String[] titel, int[] ausleihen, int grenze) {
    String[] zwischen = new String[titel.length];
    int k = 0;
    for (int i = 0; i < titel.length; i++) {
        if (ausleihen[i] >= grenze) {
            zwischen[k] = titel[i];
            k++;
        }
    }
    String[] erg = new String[k];
    for (int j = 0; j < k; j++) {
        erg[j] = zwischen[j];
    }
    return erg;
}
```

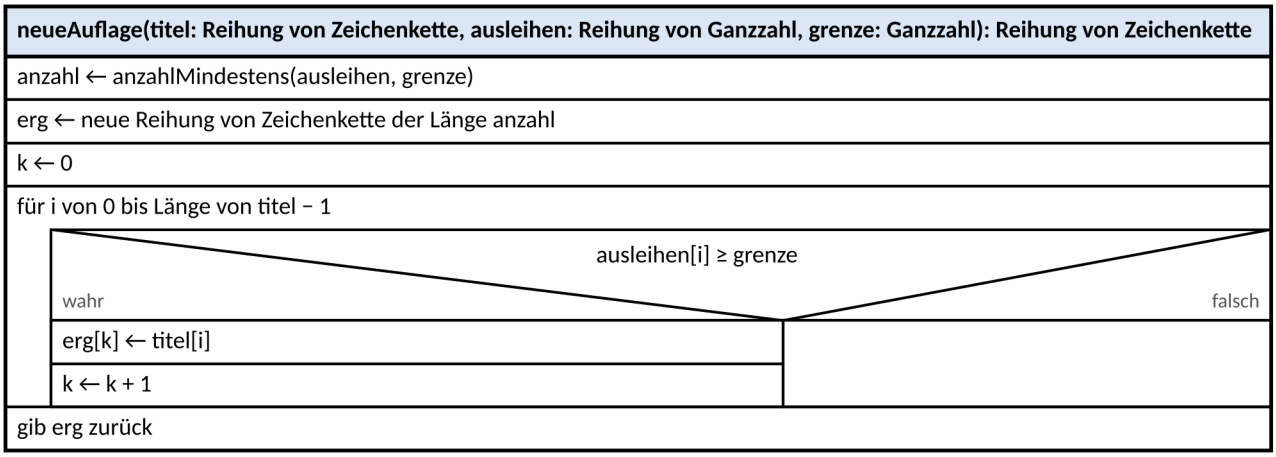


Abbildung EH.2: neueAuflage als Struktogramm (bei Abgabe als Struktogramm höchstens 3 BE)

Typische Fehler

- > statt ≥ – Bücher mit genau grenze Ausleihen fehlen (Test grenzeZaehltMit rot).
- erg mit Länge titel.length → null-Einträge am Ende.
- erg[i] statt erg[k] → ArrayIndexOutOfBoundsException.
- k++ vergessen → jeder Treffer überschreibt erg[0].
- return null statt leerer Reihung, wenn kein Buch passt.
- Rückgabetyp String statt String[] bzw. Ergebnisreihung als int[] deklariert.

f) Fehleranalyse und Korrektur der Operation durchschnitt 4 BE (I 1 · II 2 · III 1)

Musterlösung

Fehler 1 (fehlender Rückgabepfad): Ist die Reihung leer, ist die Bedingung falsch und die Operation endet ohne return. Der Compiler lehnt die Methode deshalb mit „missing return statement“ ab. Korrektur: return 0.0 für die leere Reihung.

Fehler 2 (Off-by-one): Die Schleife beginnt bei i = 1, ausleihen[0] wird nie addiert. Mit Abbildung 1 ist summe = 62 statt 74.

Fehler 3 (ganzzahlige Division): summe und ausleihen.length sind Ganzzahlen, summe / ausleihen.length wird ohne Rest gerechnet; erst das Ergebnis wird in eine Kommazahl umgewandelt. Mit Abbildung 1 ergibt sich 62 / 8 = 7 → **7.0** statt **9.25** (allein wirkt Fehler 2 mit 7.75, Fehler 3 mit 9.0). Alle Werte durch Ausführen geprüft.

Korrigierte Implementierung:

```
public static double durchschnitt(int[] ausleihen) {
    if (ausleihen.length == 0) {
        return 0.0;
    }
    int summe = 0;
    for (int i = 0; i < ausleihen.length; i++) {
        summe = summe + ausleihen[i];
    }
    return (double) summe / ausleihen.length;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Identifikation der drei Fehler	I	1	☐
Auswirkung des fehlenden Rückgabepfads (Compilerfehler, leere Reihung)	II	1	☐
Auswirkungen von Off-by-one und ganzzahliger Division mit Werten aus Abbildung 1	II	1	☐
Vollständig korrigierte Implementierung	III	1	☐

Alternativen / Teilpunkte: Gleichwertige Korrekturen der Division: summe als double deklarieren; 1.0 * summe / ausleihen.length. Nicht ausreichend: (double) (summe / ausleihen.length) – der Cast wirkt erst nach der ganzzahligen Division. Die Struktur mit if … else und return in beiden Zweigen ist ebenso richtig.

Typische Fehler

- Fehlender Rückgabepfad übersehen, weil „eine leere Reihung ja nicht vorkommt“.
- Korrektur (double) (summe / ausleihen.length).
- Off-by-one als „das letzte Element fehlt“ beschrieben.
- Nur return 0.0 ergänzt – die beiden logischen Fehler bleiben (Ergebnis 7.0).

Anhang: Papier-Compiler-Workspace zu e)

Für den Papier-Compiler legst du in der Online-IDE einen Workspace „W0 Boss 0 – Papier-Compiler“ mit der Klasse Bibliothek (Gerüst unten, neueAuflage mit leerem Rumpf und return null;) und der Testklasse BossTest an und verteilst ihn über „An Klasse austeilen“. Die Lernenden tippen ihre Handschrift **unverändert** in den Rumpf ab und starten die Tests im Reiter „Testrunner“. Erst danach dürfen Fehler korrigiert werden (dann ohne Erster-Versuch-Bonus).

Vor Einsatz prüfen

JUnit-Dialekt der Online-IDE (parameterloser Konstruktor der Testklasse, @Test, globale assertEquals/assertTrue mit Meldung als letztem Argument) vor dem ersten Einsatz mit einem Probedurchlauf testen – insbesondere den Aufruf der Klassenoperation über Bibliothek.neueAuflage(...), Reihungs-Literale new String[] {...} und assertEquals mit Zeichenketten. Die Hilfsmethoden titel(), ausleihen() und text(...) der Testklasse sind private und ohne @Test – prüfen, dass der Testrunner sie nicht als Tests aufführt. Die Bibliothek „niedersachsen“ wird nicht benötigt.

Klasse Bibliothek (Gerüst für den Workspace; sterne, sterneSumme und anzahlMindestens sind vorhanden)

```
public class Bibliothek {

    public static int sterne(int n) {
        if (n == 0) {
            return 0;
        }
        int s = n / 5 + 1;
        if (s > 5) {
            return 5;
        }
        return s;
    }

    public static int sterneSumme(int[] ausleihen, int start, int ende) {
        int summe = 0;
        for (int i = start; i <= ende; i++) {
            summe = summe + sterne(ausleihen[i]);
        }
        return summe;
    }

    public static int anzahlMindestens(int[] ausleihen, int grenze) {
        int anzahl = 0;
        for (int i = 0; i < ausleihen.length; i++) {
            if (ausleihen[i] >= grenze) {
                anzahl++;
            }
        }
        return anzahl;
    }

    public static String[] neueAuflage(String[] titel, int[] ausleihen, int grenze) {
        // HIER die Handschrift aus Aufgabe e) UNVERAENDERT abtippen
        return null;
    }
}
```

Testklasse BossTest (8 Tests; mit der Musterlösung und beiden Alternativen alle grün, typische Fehler werden erkannt – geprüft mit javac, siehe Tabelle)

```
public class BossTest {

    public BossTest() {
    }

    private String[] titel() {
        return new String[] {"Feuerkunde", "Trankbuch", "Runenlehre", "Sternkarten",
                "Kräuterfibel", "Drachenrecht", "Zahlenmagie", "Nebelpfade"};
    }

    private int[] ausleihen() {
        return new int[] {12, 0, 7, 25, 3, 0, 18, 9};
    }

    private String text(String[] r) {
        String s = "";
        for (int i = 0; i < r.length; i++) {
            s = s + r[i] + " ";
        }
        return s;
    }

    @Test
    public void beispielGrenze9() {
        String[] e = Bibliothek.neueAuflage(titel(), ausleihen(), 9);
        assertEquals(4, e.length, "Laenge 4 erwartet");
        assertEquals("Feuerkunde Sternkarten Zahlenmagie Nebelpfade ", text(e),
                "Titel in der Reihenfolge von titel erwartet");
    }

    @Test
    public void grenzeZaehltMit() {
        String[] e = Bibliothek.neueAuflage(titel(), ausleihen(), 25);
        assertEquals(1, e.length, "mindestens 25: genau Sternkarten");
        assertEquals("Sternkarten", e[0], "Sternkarten (25) zaehlt mit");
    }

    @Test
    public void keinTrefferLiefertLeereReihung() {
        String[] e = Bibliothek.neueAuflage(titel(), ausleihen(), 100);
        assertTrue(e != null, "nicht null, sondern eine leere Reihung erwartet");
        assertEquals(0, e.length, "Laenge 0 erwartet");
    }

    @Test
    public void leereReihungen() {
        String[] e = Bibliothek.neueAuflage(new String[0], new int[0], 5);
        assertTrue(e != null, "nicht null erwartet");
        assertEquals(0, e.length, "Laenge 0 erwartet");
    }

    @Test
    public void grenzeNullAlleTitel() {
        String[] e = Bibliothek.neueAuflage(titel(), ausleihen(), 0);
        assertEquals(8, e.length, "alle 8 Titel erwartet");
        assertEquals("Feuerkunde", e[0], "erster Titel");
        assertEquals("Nebelpfade", e[7], "letzter Titel");
    }
```

```
    @Test
    public void originaleBleibenUnveraendert() {
        String[] t = titel();
        int[] a = ausleihen();
        Bibliothek.neueAuflage(t, a, 9);
        assertEquals(text(titel()), text(t), "titel darf nicht veraendert werden");
        assertEquals(25, a[3], "ausleihen darf nicht veraendert werden");
    }

    @Test
    public void ersterUndLetzterTreffer() {
        String[] t = {"A", "B", "C"};
        int[] a = {5, 1, 5};
        String[] e = Bibliothek.neueAuflage(t, a, 5);
        assertEquals("A C ", text(e), "A und C erwartet");
    }

    @Test
    public void trefferNurHinten() {
        String[] t = {"X", "Y", "Z"};
        int[] a = {1, 9, 9};
        String[] e = Bibliothek.neueAuflage(t, a, 2);
        assertEquals("Y Z ", text(e), "Y und Z erwartet");
    }
}
```

Kontrolle der Testklasse mit typischen Fehlvarianten (Mutanten)

Variante von neueAuflage	Tests grün
Musterlösung (mit Hilfsmethode anzahlMindestens)	8/8
Alternative (1): eigene Zählschleife statt anzahlMindestens	8/8
Alternative (2): Zwischenreihung, danach in passende Länge kopieren	8/8
> statt >= (Bücher mit genau grenze Ausleihen fehlen)	4/8
neue Reihung mit Länge titel.length (null-Einträge)	3/8
erg[i] statt erg[k] (Index der Quelle verwendet)	3/8
k++ vergessen (alle Treffer landen in erg[0])	4/8
null statt leerer Reihung, wenn kein Buch passt	6/8
Schleife beginnt bei i = 1	5/8
Original verändert (Treffer in titel nach vorne geschrieben)	7/8
leerer Rumpf (Gerüst, return null;)	1/8